# Visão Geral e Requisitos do Financitas

## 1. Visão Geral do Sistema

O Financitas é uma aplicação web local focada em inteligência financeira pessoal. Ele automatiza a leitura, categorização e análise de faturas de cartão de crédito (inicialmente focado no formato CSV do Nubank) utilizando Processamento de Linguagem Natural (NLP) e Estatística Aplicada. O objetivo central é fornecer um panorama claro do fluxo de caixa, prever gastos futuros, identificar anomalias estatísticas de consumo e gerenciar metas financeiras com alta previsibilidade e total privacidade de dados.

## 2. Arquitetura e Stack Tecnológica

A arquitetura foi desenhada para ser modular, conteinerizada e separar claramente as responsabilidades de interface e processamento de dados, garantindo portabilidade em formato **.zip** .

- **Frontend (Interface): HTML**, **CSS** e **JavaScript** puros (interatividade e consumo da API via *Fetch API*).
- **Backend (API REST e Motor de IA):** Python utilizando **FastAPI** para gerenciar as rotas. Processamento de dados com **Pandas** e motor de machine learning com **Scikit-Learn**.
- **Banco de Dados: PostgreSQL**.
- **Infraestrutura: Docker** e **Docker Compose** (permitindo a inicialização simultânea do banco e do backend com um único comando).

## 3. Requisitos Funcionais (RF)

- **RF01 - Ingestão Inteligente de Dados:** O sistema deve processar arquivos CSV de faturas, extraindo data, descrição e valor. Deve possuir lógica interna para evitar a duplicidade de registros em envios sobrepostos do mesmo mês.
- **RF02 - Módulo de Classificação (Human-in-the-Loop):** O sistema utilizará um modelo de NLP treinado inicialmente com um "Dataset Zero". Após o upload, a interface exibirá "Gastos Classificados" (com a probabilidade de acerto visível) e "Gastos Incertos" (classificações abaixo do limiar de segurança). O usuário deve poder corrigir classificações, retroalimentando o modelo para re-treinamento.
- **RF03 - Taxonomia Estrita de Categorias:** O classificador inicial trabalhará com categorias fixas e sem subníveis para garantir precisão: Aluguel, Internet, Água, Transporte, Energia, Feira, Entretenimento, Zahy e Parcelamentos.
- **RF04 - Gestão de Parcelamentos:** O sistema utilizará Expressões Regulares (**Regex**) no backend para identificar compras parceladas (ex: 1/12). O impacto financeiro será restrito ao fluxo de caixa do mês vigente, possuindo uma visualização dedicada detalhando o valor da parcela, a fração atual e o saldo devedor restante.
- **RF05 - Painel de Receitas e Despesas:** Um campo para inserção da receita mensal será disponibilizado. O dashboard consumirá esses dados para renderizar gráficos interativos (Barra e Pizza) mostrando o impacto das despesas sobre a

receita e a distribuição por categoria (incluindo ícone, nome, cor, valor absoluto e percentual).
- **RF06 - Detecção de Anomalias:** O sistema destacará desvios de padrão financeiro, utilizando cálculos estatísticos (**Z-index**) sobre o histórico. A interface contará com um filtro para alternar entre "Gastos Únicos", "Categorias" ou "Todos", comparando o gasto atípico com a média histórica.
- **RF07 - Módulo de Previsão de Gastos (Forecasting):** Seção analítica dedicada a projetar e exibir (via gráficos e listas) a estimativa de despesas para os meses seguintes, baseada no comportamento estatístico e sazonalidade de cada categoria.
- **RF08 - Simulador de Metas Financeiras:** Ferramenta interativa e independente onde o usuário define um objetivo financeiro. O sistema cruzará a meta com os dados gerados pelo motor de Forecasting para calcular a viabilidade e sugerir o valor exato a ser retido mensalmente.

## 4. Requisitos Não-Funcionais (RNF)

- **RNF01 - Portabilidade Absoluta:** Todo o ambiente deve estar encapsulado no Docker Compose. O sistema deve ser transferível via `.zip` ou HD externo para ser executado em outros computadores (como o notebook da Riane) sem necessidade de instalações locais além do próprio Docker.
- **RNF02 - Privacidade Total:** Todos os dados bancários e modelos treinados devem permanecer locais, armazenados estritamente no contêiner do PostgreSQL da máquina em execução.
- **RNF03 - Resiliência no "Cold Start":** O processo de treinamento inicial da IA deve descartar dados históricos ruidosos ou não-categorizados (como transações genéricas via Pix), garantindo que o classificador aprenda apenas com informações de alta confiança.

## 5. Roadmap de Desenvolvimento

- **Fase 1: Fundação e Infraestrutura**
  - Criação do arquivo `docker-compose.yml` contendo os serviços do PostgreSQL e a base do FastAPI.
  - Modelagem relacional do banco de dados e execução dos scripts SQL para criação das tabelas (faturas, transações, categorias, etc.).
- **Fase 2: Backend e Processamento (API)**
  - Desenvolvimento dos scripts Python (Pandas) para limpeza do CSV Nubank e isolamento de parcelamentos via Regex.
  - Filtragem manual do Dataset Zero e treinamento do modelo classificador base (Scikit-Learn).
  - Criação dos endpoints no FastAPI (`/upload`, `/predict`, `/confirm`).
- **Fase 3: Frontend e Integração**
  - Construção do layout com HTML e CSS (menus, cards e tabelas).
  - Implementação do JavaScript com `fetch` para enviar o CSV ao backend e popular as listas expansíveis de classificação.
  - Integração de bibliotecas gráficas (ex: Chart.js) para gerar os dashboards de consumo.

- **Fase 4: Inteligência Estatística Avançada**
    - Implementação da lógica matemática no backend para detecção de anomalias (cálculo de média e desvio padrão).
    - Criação do motor de Forecasting para projeção de gastos futuros.
    - Conexão do Simulador de Metas no frontend consumindo as previsões do backend, finalizando com testes de fluxo e portabilidade via `.zip`.